

FUNDAMENTO TEÓRICO

Un procedimiento almacenado son sentencias SQL encapsuladas dentro de la sentencia `CREATE PROCEDURE`. El procedimiento almacenado puede contener declaraciones condicionales como `IF` o `CASE`, ciclos, e incluso ejecutar otros procedimientos almacenados o funciones para modularizar el código.

Beneficios de los procedimientos almacenados

Reducir el tráfico de red

Se encapsulan varias sentencias SQL dentro de un procedimiento almacenado. Cuando se ejecuta, en lugar de enviar múltiples consultas SQL, solamente se envía el nombre del procedimiento y sus parámetros.

Fácil mantenimiento

Los procedimientos almacenados son reutilizables. La lógica de negocio puede implementarse una sola vez y ser utilizada por diferentes módulos o aplicaciones, logrando mayor consistencia en la base de datos.

Seguridad

Los procedimientos almacenados son más seguros que las consultas AdHoc. Se puede otorgar permiso únicamente para ejecutar el procedimiento sin dar acceso directo a las tablas. Además, ayudan a evitar ataques de inyección SQL.

DESVENTAJAS DE LOS STORE PROCEDURES

- Pueden ser difíciles de mantener cuando el código es muy grande.
- La depuración de errores es más complicada.
- Generan dependencia del gestor de base de datos utilizado.
- Procedimientos mal optimizados pueden afectar el rendimiento.
- Si un procedimiento falla, puede afectar varios módulos o aplicaciones que lo utilizan.

Procedimiento 1 - actualiza_impuesto_clave

1. Crear procedimiento

```
1 CREATE PROCEDURE actualizar_impuesto_clave
2   p_clave INT,
3   p_nuevo_impuesto INT
4 AS
5 BEGIN
6   UPDATE "Materiales"
7   SET impuesto = p_nuevo_impuesto
8   WHERE clave = p_clave;
9 COMMIT;
```

2. Consulta inicial

```
1 SELECT *
2 FROM "Materiales"
3 WHERE clave = 1000;
```

clave	descripcion	precio	impuesto	PorcentajeImp
1000	Varilla 3/16	100	10	2.00

3. Ejecutar procedimiento

```
1 CALL actualizar_impuesto_clave(1000, 22);
```

clave	descripcion	precio	impuesto	PorcentajeImpuesto
1000	Varilla 3/16	100	22	2.00

4. Consulta Final

```
1 SELECT *
2 FROM "Materiales"
3 WHERE clave = 1000;
```

clave	descripcion	precio	impuesto	PorcentajeImp
1000	Varilla 3/16	100	22	2.00

Procedimiento 2 - actualiza precio

1. Crear procedimiento

```
CREATE OR REPLACE PROCEDURE actualiza_precio(  
    p_clave INT,  
    p_nuevo_precio NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE "Materiales"  
    SET precio = p_nuevo_precio  
    WHERE clave = p_clave;  
    COMMIT;  
END;  
$$;
```

2. Consulta Inicial

Results Explain Chart Export ▾				
clave	descripcion	precio	impuesto	PorcentajeImp
1000	Varilla 3/16	100	22	2.00

3. Ejecutar procedimiento

```
CALL actualiza_precio(1000, 35);
```

4. Consulta final

```
SELECT *  
FROM "Materiales"  
WHERE clave = 1000;
```

clave	descripcion	precio	impuesto	PorcentajeImp
1000	Varilla 3/16	35	22	2.00

Procedimiento 3 - actualiza cantidad entregada

1. Crear procedimiento

```
CREATE OR REPLACE PROCEDURE actualiza_cantidad_entregada(  
    p_clave INT,  
    p_rfc VARCHAR,  
    p_numero INT,  
    p_nueva_cantidad INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  
    UPDATE "Entregan"  
    SET cantidad = p_nueva_cantidad  
    WHERE clave = p_clave  
    AND rfc = p_rfc  
    AND numero = p_numero;  
  
    COMMIT;  
  
END;  
$$;
```

2. Consulta inicial

```
SELECT *  
FROM "Entregan"  
WHERE clave = 1000  
AND rfc = 'AAAA800101'  
AND numero = 5019;
```

clave	rfc	numero	fecha	cantidad
1000	AAAA800101	5019	1999-07-13	254

3. Ejecutar procedimiento

```
CALL actualiza_cantidad_entregada(1000,'AAAA800101', 5019, 500);
```

4. Consulta Final

```
SELECT *  
FROM "Entregan"  
WHERE clave = 1000  
AND rfc = 'AAAA800101'  
AND numero = 5019;
```

clave	rfc	numero	fecha	cantidad
1000	AAAA800101	5019	1999-07-13	500